PL/SQL Exercises: Alzaheer Khan K

Database Schema & Reference Architecture

The solutions provided in this document are based on the following relational database schema structure:

```
-- Tables Structure
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    DOB DATE,
    Balance NUMBER,
    LastModified DATE
);

CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    AccountType VARCHAR2(20),
    Balance NUMBER,
    LastModified DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Transactions (
    TransactionID NUMBER PRIMARY KEY,
    AccountID NUMBER,
    TransactionDate DATE,
    Amount NUMBER,
    TransactionType VARCHAR2(10),
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);

CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    LoanAmount NUMBER,
    InterestRate NUMBER,
    StartDate DATE,
    EndDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

```
CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    Position VARCHAR2(50),
    Salary NUMBER,
    Department VARCHAR2(50),
    HireDate DATE
);
```

Exercise 1: Control Structures

Scenario 1: Interest Rate Discount for Senior Citizens

Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
DECLARE
    CURSOR c_senior_loans IS
        SELECT l.LoanID, l.InterestRate, c.Name
        FROM Loans l
        JOIN Customers c ON l.CustomerID = c.CustomerID
        WHERE MONTHS_BETWEEN(SYSDATE, c.DOB) / 12 > 60;
BEGIN
    FOR r_loan IN c_senior_loans LOOP
        UPDATE Loans
        SET InterestRate = InterestRate - 1
        WHERE LoanID = r_loan.LoanID;

        DBMS_OUTPUT.PUT_LINE('Applied 1% discount to Loan ID: ' || r_loan.LoanID
|| ' for Customer: ' || r_loan.Name);
    END LOOP;
    COMMIT;
END;
/
```

Scenario 2: VIP Status Promotion

Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over $10,000.

> Note: As the initial schema does not contain an `IsVIP` column, this solution includes the schema modification or assumes its existence.

```
-- Assumption: Table has been modified to support the flag
-- ALTER TABLE Customers ADD IsVIP VARCHAR2(5) DEFAULT 'FALSE';

BEGIN
    FOR r_cust IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF r_cust.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = r_cust.CustomerID;
        ELSE
            UPDATE Customers
            SET IsVIP = 'FALSE'
            WHERE CustomerID = r_cust.CustomerID;
        END IF;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('VIP status promotion processing completed
successfully.');
END;
/
```

Scenario 3: Loan Due Reminders

Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
DECLARE
    CURSOR c_due_loans IS
        SELECT l.LoanID, c.Name, l.EndDate
        FROM Loans l
        JOIN Customers c ON l.CustomerID = c.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30;
BEGIN
    FOR r_loan IN c_due_loans LOOP
        DBMS_OUTPUT.PUT_LINE('REMINDER: Hello ' || r_loan.Name || ', your Loan
(ID: ' || r_loan.LoanID || ') is due on ' || TO_CHAR(r_loan.EndDate, 'YYYY-MM-DD')
|| '. Please ensure timely payment.');
    END LOOP;
END;
/
```

Exercise 2: Error Handling

Scenario 1: Safe Fund Transfer Procedure

Question: Write a stored procedure SafeTransferFunds that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (
    p_SourceAccountID IN NUMBER,
    p_DestAccountID   IN NUMBER,
    p_Amount          IN NUMBER
) IS
    v_SourceBalance NUMBER;
    insufficient_funds EXCEPTION;
BEGIN
    -- Check source account balance
    SELECT Balance INTO v_SourceBalance FROM Accounts WHERE AccountID =
p_SourceAccountID FOR UPDATE;

    IF v_SourceBalance < p_Amount THEN
        RAISE insufficient_funds;
    END IF;

    -- Perform the debit
    UPDATE Accounts SET Balance = Balance - p_Amount, LastModified = SYSDATE WHERE
AccountID = p_SourceAccountID;

    -- Perform the credit
    UPDATE Accounts SET Balance = Balance + p_Amount, LastModified = SYSDATE WHERE
AccountID = p_DestAccountID;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer of ' || p_Amount || ' completed
successfully.');
EXCEPTION
    WHEN insufficient_funds THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Transfer failed due to insufficient funds in
Account ID: ' || p_SourceAccountID);
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: One or both Account IDs do not exist.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: An unexpected error occurred: ' || SQLERRM);
END;
/
```

Scenario 2: Employee Salary Update with Validation

Question: Write a stored procedure UpdateSalary that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```
CREATE OR REPLACE PROCEDURE UpdateSalary (
    p_EmployeeID IN NUMBER,
    p_Percentage IN NUMBER
) IS
    v_CurrentSalary NUMBER;
BEGIN
    -- Verify if employee exists
    SELECT Salary INTO v_CurrentSalary FROM Employees WHERE EmployeeID =
p_EmployeeID FOR UPDATE;

    -- Update salary
    UPDATE Employees
    SET Salary = Salary * (1 + p_Percentage / 100)
    WHERE EmployeeID = p_EmployeeID;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salary updated successfully for Employee ID: ' ||
p_EmployeeID);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Employee ID ' || p_EmployeeID || ' does not
exist.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: Failed to update salary. Details: ' ||
SQLERRM);
END;
/
```

Scenario 3: Primary Key Conflict Prevention

Question: Write a stored procedure AddNewCustomer that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
CREATE OR REPLACE PROCEDURE AddNewCustomer (
    p_CustomerID IN NUMBER,
    p_Name       IN VARCHAR2,
    p_DOB        IN DATE,
    p_Balance    IN NUMBER
) IS
BEGIN
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (p_CustomerID, p_Name, p_DOB, p_Balance, SYSDATE);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Customer ' || p_Name || ' added successfully with ID: '
|| p_CustomerID);
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: A customer with ID ' || p_CustomerID || '
already exists.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: Failed to add customer. Details: ' ||
SQLERRM);
END;
/
```

Exercise 3: Stored Procedures

Scenario 1: Monthly Interest Batch Processing

Question: Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance * 1.01,
        LastModified = SYSDATE
    WHERE AccountType = 'Savings';

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly 1% interest applied to all Savings accounts.');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error processing monthly interest: ' || SQLERRM);
END;
/
```

Scenario 2: Departmental Bonus Allocation

Question: Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_Department IN VARCHAR2,
    p_BonusPct   IN NUMBER
) IS
BEGIN
    UPDATE Employees
    SET Salary = Salary * (1 + p_BonusPct / 100)
    WHERE Department = p_Department;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Bonus updated for all employees in department: ' ||
p_Department);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error updating departmental bonuses: ' || SQLERRM);
END;
/
```

Scenario 3: Standard Fund Transfer Procedure

Question: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
CREATE OR REPLACE PROCEDURE TransferFunds (
    p_SourceAcc IN NUMBER,
    p_DestAcc  IN  NUMBER,
    p_Amount   IN NUMBER
) IS
    v_Balance NUMBER;
BEGIN
    SELECT Balance INTO v_Balance FROM Accounts WHERE AccountID = p_SourceAcc FOR
UPDATE;

    IF v_Balance >= p_Amount THEN
        UPDATE Accounts SET Balance = Balance - p_Amount, LastModified = SYSDATE
WHERE AccountID = p_SourceAcc;
        UPDATE Accounts SET Balance = Balance + p_Amount, LastModified = SYSDATE
WHERE AccountID = p_DestAcc;
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Successfully transferred ' || p_Amount || ' from '
|| p_SourceAcc || ' to ' || p_DestAcc);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Transfer aborted: Insufficient funds in Account ' ||
p_SourceAcc);
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: One or both account numbers do not exist.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
END;
/
```

Exercise 4: Functions

Scenario 1: Age Calculation Utility

Question: Write a function CalculateAge that takes a customer's date of birth as input and returns their age in years.

```
CREATE OR REPLACE FUNCTION CalculateAge (
    p_DOB IN DATE
) RETURN NUMBER IS
    v_Age NUMBER;
BEGIN
    v_Age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_DOB) / 12);
    RETURN v_Age;
END;
/
```

Scenario 2: Equated Monthly Installment (EMI) Calculator

Question: Write a function CalculateMonthlyInstallment that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment (
    p_LoanAmount    IN NUMBER,
    p_InterestRate IN NUMBER, -- Annual percentage rate (e.g., 5 for 5%)
    p_DurationYrs  IN NUMBER
) RETURN NUMBER IS
    v_MonthlyRate NUMBER;
    v_TotalMonths NUMBER;
    v_EMI         NUMBER;
BEGIN
    v_MonthlyRate := (p_InterestRate / 100) / 12;
    v_TotalMonths := p_DurationYrs * 12;

    IF v_MonthlyRate = 0 THEN
        v_EMI := p_LoanAmount / v_TotalMonths;
    ELSE
        v_EMI := (p_LoanAmount * v_MonthlyRate * POWER(1 + v_MonthlyRate,
v_TotalMonths)) /
                 (POWER(1 + v_MonthlyRate, v_TotalMonths) - 1);
    END IF;

    RETURN ROUND(v_EMI, 2);
END;
/
```

Scenario 3: Balance Verification Rule

Question: Write a function HasSufficientBalance that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

```
CREATE OR REPLACE FUNCTION HasSufficientBalance (
    p_AccountID IN NUMBER,
    p_Amount    IN NUMBER
) RETURN BOOLEAN IS
    v_Balance NUMBER;
BEGIN
    SELECT Balance INTO v_Balance FROM Accounts WHERE AccountID = p_AccountID;

    IF v_Balance >= p_Amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/
```

Exercise 5: Triggers

Scenario 1: Auto-Audit Modification Timestamp

Question: Write a trigger UpdateCustomerLastModified that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
```

Scenario 2: Centralized Transaction Audit Logging

Question: Write a trigger LogTransaction that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

> Note: This solution assumes a standard `AuditLog` table structure to store the logged history.

```
-- Supporting structure assumed:
-- CREATE TABLE AuditLog (LogID NUMBER GENERATED ALWAYS AS IDENTITY, TransactionID
NUMBER, LogDate DATE);

CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (TransactionID, LogDate)
    VALUES (:NEW.TransactionID, SYSDATE);
END;
/
```

Scenario 3: Business Rule Enforcement Engine

Question: Write a trigger CheckTransactionRules that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_CurrentBalance NUMBER;
BEGIN
    -- For Withdrawal, check sufficiency
    IF :NEW.TransactionType = 'Withdrawal' THEN
        SELECT Balance INTO v_CurrentBalance FROM Accounts WHERE AccountID
= :NEW.AccountID;
        IF :NEW.Amount > v_CurrentBalance THEN
            RAISE_APPLICATION_ERROR(-20001, 'Transaction Aborted: Withdrawal
amount exceeds current account balance.');
        END IF;
    -- For Deposit, ensure it is a positive value
    ELSIF :NEW.TransactionType = 'Deposit' THEN
        IF :NEW.Amount <= 0 THEN
            RAISE_APPLICATION_ERROR(-20002, 'Transaction Aborted: Deposit amount
must be greater than zero.');
        END IF;
    END IF;
END;
/
```

Exercise 6: Cursors

Scenario 1: Monthly Statement Generation

Question: Write a PL/SQL block using an explicit cursor GenerateMonthlyStatements that retrieves all transactions for the current month and prints a statement for each customer.

```
DECLARE
    CURSOR GenerateMonthlyStatements IS
        SELECT c.Name, t.AccountID, t.TransactionID, t.Amount, t.TransactionType,
t.TransactionDate
        FROM Transactions t
        JOIN Accounts a ON t.AccountID = a.AccountID
        JOIN Customers c ON a.CustomerID = c.CustomerID
        WHERE TRUNC(t.TransactionDate, 'MM') = TRUNC(SYSDATE, 'MM');
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- MONTHLY TRANSACTION STATEMENTS ---');
    FOR r_stmt IN GenerateMonthlyStatements LOOP
        DBMS_OUTPUT.PUT_LINE('Customer: ' || r_stmt.Name || ' | Acc: ' ||
r_stmt.AccountID ||
                             ' | Txn ID: ' || r_stmt.TransactionID || ' | Type: '
|| r_stmt.TransactionType ||
                             ' | Amt: $' || r_stmt.Amount || ' | Date: ' ||
TO_CHAR(r_stmt.TransactionDate, 'YYYY-MM-DD'));
    END LOOP;
END;
/
```

Scenario 2: Annual Maintenance Fee Deduction

Question: Write a PL/SQL block using an explicit cursor ApplyAnnualFee that deducts an annual maintenance fee from the balance of all accounts.

```
DECLARE
    v_MaintenanceFee CONSTANT NUMBER := 50.00;
    CURSOR ApplyAnnualFee IS
        SELECT AccountID, Balance FROM Accounts FOR UPDATE;
BEGIN
    FOR r_acc IN ApplyAnnualFee LOOP
        UPDATE Accounts
        SET Balance = Balance - v_MaintenanceFee,
            LastModified = SYSDATE
        WHERE CURRENT OF ApplyAnnualFee;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Annual maintenance fee applied to all accounts
successfully.');
END;
/
```

Scenario 3: Policy-Driven Loan Interest Update

Question: Write a PL/SQL block using an explicit cursor UpdateLoanInterestRates that fetches all loans and updates their interest rates based on the new policy.

> Note: This scenario assumes a 0.5% interest adjustment policy condition as an example implementation.

```
DECLARE
    CURSOR UpdateLoanInterestRates IS
        SELECT LoanID, InterestRate FROM Loans FOR UPDATE;
BEGIN
    FOR r_loan IN UpdateLoanInterestRates LOOP
        -- Example Policy rule: Adjust rates dynamically or by fixed policy
coefficient
        UPDATE Loans
        SET InterestRate = InterestRate + 0.5
        WHERE CURRENT OF UpdateLoanInterestRates;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Loan interest rates updated successfully matching the
new policy.');
END;
/
```

Exercise 7: Packages

Scenario 1: Customer Management Suite

Question: Create a package CustomerManagement with procedures for adding a new customer, updating customer details, and a function to get customer balance.

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddNewCustomer(p_CustID NUMBER, p_Name VARCHAR2, p_DOB DATE,
p_Balance NUMBER);
    PROCEDURE UpdateCustomerDetails(p_CustID NUMBER, p_Name VARCHAR2);
    FUNCTION GetCustomerBalance(p_CustID NUMBER) RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
    PROCEDURE AddNewCustomer(p_CustID NUMBER, p_Name VARCHAR2, p_DOB DATE,
p_Balance NUMBER) IS
    BEGIN
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
        VALUES (p_CustID, p_Name, p_DOB, p_Balance, SYSDATE);
    END AddNewCustomer;

    PROCEDURE UpdateCustomerDetails(p_CustID NUMBER, p_Name VARCHAR2) IS
    BEGIN
        UPDATE Customers SET Name = p_Name, LastModified = SYSDATE WHERE
CustomerID = p_CustID;
    END UpdateCustomerDetails;

    FUNCTION GetCustomerBalance(p_CustID NUMBER) RETURN NUMBER IS
        v_Bal NUMBER;
    BEGIN
        SELECT Balance INTO v_Bal FROM Customers WHERE CustomerID = p_CustID;
        RETURN v_Bal;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN RETURN 0;
    END GetCustomerBalance;
END CustomerManagement;
/
```

Scenario 2: Employee Management Suite

Question: Write a package EmployeeManagement with procedures to hire new employees, update employee details, and a function to calculate annual salary.

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee(p_EmpID NUMBER, p_Name VARCHAR2, p_Pos VARCHAR2, p_Sal
NUMBER, p_Dept VARCHAR2);
    PROCEDURE UpdateEmployee(p_EmpID NUMBER, p_Pos VARCHAR2, p_Sal NUMBER);
    FUNCTION GetAnnualSalary(p_EmpID NUMBER) RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
    PROCEDURE HireEmployee(p_EmpID NUMBER, p_Name VARCHAR2, p_Pos VARCHAR2, p_Sal
NUMBER, p_Dept VARCHAR2) IS
    BEGIN
        INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department,
HireDate)
        VALUES (p_EmpID, p_Name, p_Pos, p_Sal, p_Dept, SYSDATE);
    END HireEmployee;

    PROCEDURE UpdateEmployee(p_EmpID NUMBER, p_Pos VARCHAR2, p_Sal NUMBER) IS
    BEGIN
        UPDATE Employees SET Position = p_Pos, Salary = p_Sal WHERE EmployeeID =
p_EmpID;
    END UpdateEmployee;

    FUNCTION GetAnnualSalary(p_EmpID NUMBER) RETURN NUMBER IS
        v_MonthlySal NUMBER;
    BEGIN
        SELECT Salary INTO v_MonthlySal FROM Employees WHERE EmployeeID = p_EmpID;
        RETURN v_MonthlySal * 12;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN RETURN 0;
    END GetAnnualSalary;
END EmployeeManagement;
/
```

Scenario 3: Account Operations Suite

Question: Create a package AccountOperations with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

```
CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount(p_AccID NUMBER, p_CustID NUMBER, p_AccType VARCHAR2,
p_InitialBal NUMBER);
    PROCEDURE CloseAccount(p_AccID NUMBER);
    FUNCTION GetTotalBalance(p_CustID NUMBER) RETURN NUMBER;
END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS
    PROCEDURE OpenAccount(p_AccID NUMBER, p_CustID NUMBER, p_AccType VARCHAR2,
p_InitialBal NUMBER) IS
    BEGIN
        INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance,
LastModified)
        VALUES (p_AccID, p_CustID, p_AccType, p_InitialBal, SYSDATE);
    END OpenAccount;

    PROCEDURE CloseAccount(p_AccID NUMBER) IS
    BEGIN
        DELETE FROM Accounts WHERE AccountID = p_AccID;
    END CloseAccount;

    FUNCTION GetTotalBalance(p_CustID NUMBER) RETURN NUMBER IS
        v_TotalBal NUMBER;
    BEGIN
        SELECT SUM(Balance) INTO v_TotalBal FROM Accounts WHERE CustomerID =
p_CustID;
        RETURN NVL(v_TotalBal, 0);
    END GetTotalBalance;
END AccountOperations;
/
```